

## RESEARCH ARTICLE

# Automated Microservices Identification Through Business Process Analysis: A Semantic-Driven Clustering Approach

IDRIS OUMOUSA<sup>1</sup> AND RAJAA SAIDI<sup>1</sup>

SI2M Laboratory, National Institute of Statistics and Applied Economics (INSEA), Rabat 10112, Morocco

Corresponding author: Idris Oumoussa (ioumoussa@insea.ac.ma)

**ABSTRACT** The rise of microservice architectures has further evolved software development practices, building upon the foundation established by web services in breaking down monolithic systems, while offering more finely detailed, coherent, and loosely interconnected services. Despite the transformative potential of microservices, the challenge of effectively pinpointing microservices that meet an organization's specific requirements remains a difficult task. This paper presents an extended approach for automating microservices identification by leveraging business processes (BPs). Addressing limitations in existing microservices identification techniques, this approach utilizes advanced Natural Language Processing (NLP) techniques—such as Named Entity Recognition (NER) and semantic analysis—to capture dependencies and align boundaries with business logic. This methodology applies unsupervised clustering algorithms, including K-means, K-medoids, and DBSCAN, to generate well-defined microservices by analyzing semantic similarity across BP activities. Evaluation on real-world case studies, such as the Bicing bicycle rental system, Cargo Tracking, and JPetStore, demonstrates its effectiveness, yielding high cohesion, low coupling, and superior granularity in microservices decomposition compared to traditional methods. Metrics such as the Silhouette Index, Afferent Coupling, and Instability further validate its performance. By automating microservices identification with a BP-centered framework, this approach aligns technical architecture with organizational goals, advancing both agility and scalability in software systems. This work marks a step forward in optimizing system modularity and provides a foundation for future scalability improvements.

**INDEX TERMS** Microservices, micro-services granularity, business processes, NLP, clustering, semantic similarity, micro-services decompositions, software architecture.

## I. INTRODUCTION

The ever-increasing complexity and size of monolithic software systems pose significant challenges in terms of maintainability, extensibility, and adaptation to evolving market demands [1], [2]. As a result, enterprises face difficulties in keeping pace with changing customer requirements and the introduction of new products by competitors. Decomposing legacy applications into microservices has emerged as a crucial strategy for modern enterprises to maintain a competitive

edge [3]. Microservices architecture, characterized by the development of fine-grained, loosely coupled services, offers a promising solution to the limitations of monolithic systems [4]. By breaking down monolithic applications into smaller, independent services, microservices architecture enhances system flexibility, scalability, and deployability. This modular approach simplifies system management and enables swift adaptation to changing requirements.

When migrating a monolithic system to microservices, identifying the appropriate microservice boundaries is a critical and challenging task. Three primary strategies are commonly employed: top-down, bottom-up, or hybrid [5].

The associate editor coordinating the review of this manuscript and approving it for publication was Hongwei Du.

The top-down approach is a forward engineering process that involves analyzing high-level artifacts of the monolithic system to identify potential microservice candidates. In contrast, the bottom-up strategy is a reverse engineering technique that focuses on identifying reusable components within the monolithic system that can be repurposed as microservices. The hybrid method combines both approaches, leveraging reusable components while building additional microservices components in accordance with given specifications.

Microservices identification is a critical ingredient of all three migration methodologies, as we systematically review it within the scope of transitioning from monolithic systems [6]. The objective is to prevent system designers from being disgruntled with microservices, which might hinder their adoption. In fact, this is what we did and demonstrated in [7], where we introduced an initial framework for automating microservices identification using business processes (BPs) as a guiding principle. That research laid the foundation for leveraging BPs to inform the decomposition of monolithic systems into microservices. As far as we are aware, only [8], [9] have presented approaches for identifying microservices based on BPs, whereas other studies employ source codes [10], legacy databases [11], User Stories [12], and UML class diagrams [13]. At the core of our approach, which we extend in this paper, are three key elements: first, activities in BPs that capture the core operations of an organization, defining who performs what tasks, where, when, and why; second, semantic models that describe the relationships between these activities; and finally, the unsupervised clustering of activities based on their semantic and functional similarities. This technique allowed us to identify activities that exhibit high cohesion and low coupling, characteristics essential for effective microservice decomposition.

This paper extends the approach we discussed in [7] from three major perspectives. First, we introduce improvements to the microservice identification pipeline, including enhanced NLP techniques [14] for more precise semantic analysis and the integration of advanced clustering algorithms. These changes not only improve the accuracy of microservice identification but also make the process more scalable and adaptable. The improved approach was evaluated on two additional case studies, Cargo Tracking and JPetStore [13], [15], to demonstrate its robustness in real-world scenarios. These case studies enable a more comprehensive comparison with existing techniques, offering further evidence of the approach's practical applicability and its advantages over other methods. Additionally, we have expanded the comparison with related works to offer a more in-depth analysis of where the method stands in relation to other microservice identification techniques. This extended review of the literature allows us to highlight both the unique contributions and limitations of the approach, situating it within the broader landscape of automated system decomposition strategies.

This paper is organized as follows: Section II situates our work in relation to prior research, highlighting

its contributions and characteristics. Section III provides the foundational concepts and motivation for the work. Section IV outlines the proposed methodology for identifying microservices from BPs. Section V delves into the clustering algorithms that underpin the identification strategy. Section VI concludes the paper by summarizing the findings and discussing potential avenues for future research.

## II. RELATED WORK

### A. EXISTING APPROACHES FOR IDENTIFYING MICROSERVICES

The limitations of monolithic systems and the potential of microservices to overcome these challenges have been extensively documented in the literature [16]. In this section, we provide an overview of existing approaches for microservice identification, a critical step in the transition to microservice-based architectures.

In [8], the authors propose a multi-model approach for extracting microservices from BPs. While their approach utilizes various models to capture different aspects of BP interactions, it faces certain limitations. The control scheme focuses solely on activity execution connections, potentially overlooking broader dependencies that span multiple activities. The data dependency model, while useful for identifying data-related linkages, may not adequately capture the behavioral and semantic relationships between activities. Additionally, the semantic model, though intended to examine similarities and patterns within BPs, may not be granular enough to effectively differentiate between tightly coupled and loosely coupled activities. These limitations could potentially lead to the identification of microservices with suboptimal cohesion and coupling.

In [9], the approach utilizes a single dependence matrix and conventional clustering algorithms to identify microservices from BPs. This approach, while laying the foundation for subsequent work, faces limitations due to its simplified representation of BP interactions and reliance on basic clustering techniques.

In [17], the work introduces SMSR (Scenario-based MicroService Retrieval), an approach that recommends suitable microservices based on user-written BDD (Behavior-Driven Development) test scenarios. While SMSR employs word2vec to enhance microservice retrieval accuracy, it faces certain limitations. Its reliance on word2vec may overlook the broader semantic context of BDD scenarios, potentially leading to the retrieval of irrelevant or suboptimal microservices. Additionally, the approach's focus on filtering and ranking services based on similarity scores may not adequately consider other factors that influence microservice selection, such as non-functional requirements and deployment considerations.

In [18], the presented technique identifies microservice candidates based on functionality analysis of execution traces. While this approach provides insights into the implicit and explicit behavior of programs, it faces limitations due

to its reliance on the quality of execution traces. The effectiveness of the approach is highly dependent on the comprehensiveness and correctness of the test cases used to generate the traces. Inaccurate or incomplete test cases could lead to the identification of suboptimal or erroneous microservice candidates.

In [12], the study proposes the Microservices Backlog (MB), a semi-automatic approach for determining microservice granularity during the design phase. While the approach considers factors such as semantic similarity, complexity, cohesion, coupling, and inter-microservice interactions, it faces limitations due to its reliance on genetic programming. The effectiveness of this approach is highly dependent on the tuning of genetic programming parameters, which can be a complex and time-consuming process. Additionally, the approach may not adequately capture the evolving nature of business requirements and may not be suitable for dynamic microservices architectures.

In [19], the strategy proposed relies on identifying stored procedures that encapsulate business rules to identify microservice candidates. While this approach is straightforward, it faces several limitations. Its reliance on stored procedures may not adequately capture the broader context of business operations and may overlook important dependencies between microservices. Additionally, the manual process of determining requirements and delegating tasks to experts can be time-consuming and subjective, potentially leading to inconsistencies in microservice identification.

In [20], the discussed dataflow-driven method utilizes Data Flow Diagrams (DFDs) to identify microservice candidates. While this approach provides insights into the data flow within business processes, it faces limitations due to its simplified representation of BP interactions and reliance on basic clustering techniques. The use of DFDs may not adequately capture the complex interactions between activities and may overlook important semantic dependencies. Additionally, the approach's focus on data repositories as the primary basis for clustering may not effectively identify microservices with strong cohesion and coupling.

In [21], the presented approach identifies microservices based on a dependency graph constructed from interfaces, business functions, and database tables. While this approach provides a structural view of the monolithic system, it faces limitations due to its simplified representation of BP interactions and its reliance on static dependencies. The dependency graph may not adequately capture the dynamic and complex interactions between activities, potentially leading to the identification of suboptimal or fragmented microservices. Additionally, the approach's focus on database tables as the primary basis for clustering may not effectively identify microservices with strong cohesion and coupling, as it fails to consider the semantic relationships between activities.

In [13], the decomposition technique proposed relies on a semantic analysis of OpenAPI<sup>1</sup> specifications to identify

microservice candidates. While this approach leverages OpenAPI specifications to capture the structural aspects of APIs, it faces limitations due to its focus on syntactic similarities and its reliance on UML diagrams. The semantic analysis may not adequately capture the broader context of business operations and may overlook important dependencies between microservices. Additionally, the use of UML diagrams to link interfaces with OpenAPI resources can be time-consuming and error-prone, especially for complex systems.

In [22], the Service Cutter technique presented disassembles services by assigning nano-entities (data, actions, and artifacts) to each service based on coupling criteria such as cohesiveness, compatibility, constraint, and communication. While this approach emphasizes coupling characteristics for service decomposition, it faces limitations due to its reliance on manually defined coupling criteria and its focus on a limited set of nano-entities. The manual definition of coupling criteria can be subjective and error-prone, and the focus on data, actions, and artifacts may overlook other important aspects of service interactions. Additionally, the clustering approach may not adequately capture the complex dependencies between nano-entities, potentially leading to suboptimal microservices.

In [23], the proposed method partitions microservices based on a combination of functional and non-functional considerations, including security needs, scalability requirements, and technical requirements. While this approach considers a broader range of factors than purely functional approaches, it faces limitations due to its reliance on manually specified non-functional requirements and its rule-based partitioning method. The manual specification of non-functional requirements can be time-consuming and error-prone, and the rule-based partitioning method may not adequately capture the complex trade-offs between different non-functional requirements. Additionally, the approach may not be suitable for dynamic microservice architectures that require continuous adaptation to changing requirements.

## B. SYNTHESIS AND POSITIONING

Current methods for microservice identification often rely on simplified representations of BPs and involve manual analysis or predefined rules [8], [9], which can limit their effectiveness and scalability. Additionally, many approaches do not fully integrate non-functional requirements into the microservice identification process, which is crucial for ensuring the robustness and performance of the resulting services. Table 1 provides a comprehensive overview of these methodologies, detailing primary inputs, data modeling techniques, identification algorithms, assessment techniques, and performance evaluation metrics.

The approach prioritizes usability while consistently outperforming prior methods in real-world scenarios. Its effectiveness was evaluated through comprehensive analyses on three industrial applications using the Silhouette Index, Afferent Coupling, Efferent Coupling, Instability, and

<sup>1</sup><https://www.openapis.org/>

**TABLE 1. Related work on microservices identification approaches.**

| Approach     | Evaluation                                                                                                                                                                        | # Metrics | Metric Description                                                                                                                                        | Technique, Methodology Description                                                     | Input Data ( Starting points)                                         | Data Modeling                                                                   |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| [8]          | Two state-of-the-art case studies: Bike rental and Cargo Tracking                                                                                                                 | 5         | Dunn Index, Afferent Coupling, Efferent Coupling, Coupling Instability, Relational Cohesion                                                               | Collaborative clustering                                                               | Business Processes (BPs)                                              | Data, control, and semantic matrices dependencies                               |
| [9]          | One case study: Plan Approval BPMN process                                                                                                                                        | 1         | Accuracy                                                                                                                                                  | Centralized clustering                                                                 | Business Processes (BPs)                                              | Data and control matrices dependencies                                          |
| [17]         | Two Real-world scenarios : Gherkin Feature documents, and wikipedia corpus                                                                                                        | 3         | Precision/ Recall/ F1 score                                                                                                                               | Word2vec                                                                               | Scenarios                                                             | Semantic similarity between scenario and microservice vectors                   |
| [18]         | Four case studies (web applications): e-commerce, discussion board system, blogging system, blog sites                                                                            | 5         | Functional coupling (Interface number, Operation number, I-interaction number), Functional cohesion (Cohesion at message level, Cohesion at domain level) | Clustering using affinity propagation algorithm and clustering of semantically similar | Runtime logs                                                          | Connections between business logics and source code                             |
| [12]         | Two state-of-the-art case studies: Cargo Tracking, and Jpetstore. One real-life case studies : Foristom Conferences                                                               | 5         | Granularity, Coupling, Cohesion, Complexity, Calls and Requests                                                                                           | Genetic Algorithm                                                                      | User stories                                                          | Semantic similarity among entities of the user stories                          |
| [19]         | Real large scale system                                                                                                                                                           | 0         | There were no metrics, but a Proof of Concept was developed                                                                                               | Source code analysis                                                                   | System specifications<br>Recorded procedure codes                     | Discovering stored methods relevant to system needs                             |
| [20]         | One case study: Cargo Tracking                                                                                                                                                    | 4         | Afferent Coupling, Effective Coupling, Unstable Coupling, Relational Cohesion                                                                             | Rule-based algorithm                                                                   | Data flow diagram sentences                                           | Processes, data stores, and external entities dependency graph                  |
| [21]         | industrial application: Bank transactions management                                                                                                                              | 0         | -                                                                                                                                                         | Rules-based algorithm                                                                  | Objects including interfaces, business functions, and database tables | -                                                                               |
| [13]         | Three case studies: (1) online movie streaming system, (2) money transfer application combined with Kanban Board application, and (3) real-world OpenAPI specifications from apis | 2         | Precision/Recall                                                                                                                                          | Sematic similarity, Clustering K-means, DISCO                                          | OpenAPI specifications<br>Source Code                                 | Semantic similarities between capabilities based on the co-occurrence principle |
| [22]         | Two case studies: (1) cargo tracking, and (2) Trading system                                                                                                                      | 1         | Ranking of microservices candidates(Excellent, Expected, Unreasonable)                                                                                    | Centralized clustering                                                                 | Use cases, Security access groups, Entity Relationship Models, etc.   | Dependency graph between nonentities                                            |
| [23]         | One case study: online movie streaming system                                                                                                                                     | 0         | -                                                                                                                                                         | Rule-based algorithm                                                                   | Specifications for functional and non-functional requirements         | Weighting of dependencies between pairs of functions                            |
| Our Approach | Three state-of-the-art case studies: Bicing, Cargo Tracking, and JPetStore                                                                                                        | 5         | Silhouette Index, Afferent Coupling, Efferent Coupling, Coupling Instability, Relational Cohesion                                                         | NLP, NER Tag, BIO Tag<br>K-means, K-medoids, DBSCAN                                    | Business Processes (BPs)                                              | Semantic dependency matrices<br>Similarity distance                             |

Relational Cohesion metrics. These evaluations highlight the robustness and practical applicability of the approach.

By utilizing BPs as the primary input, the method ensures that microservice identification remains closely tied to the business context. The NLP techniques enhance the identification process by generating more accurate activity dependencies, while the combination of clustering algorithms strikes a balance between usability and performance, delivering superior outcomes compared to previous approaches. These advancements position the method as a significant contribution to the field, further advancing the state-of-the-art in monolithic system decomposition.

### III. FOUNDATIONS

#### A. MICORSERVICES ARCHITECTURE

Microservices architecture (MSA) has gained significant popularity in modern software engineering by breaking down traditional monolithic applications into fine-grained, independent services that can be designed, tested, and deployed individually [3]. Each microservice has a discrete responsibility and communicates with other services using lightweight protocols like asynchronous message buses [4]. Microservices can be developed independently using various frameworks and programming languages. A key aspect of MSA is functional decomposition, allowing the construction of higher-level applications by combining different services, as illustrated in Figure 1.

In terms of software quality, microservices improve interoperability, reusability, scalability, and maintainability. They can be deployed, scaled, or replaced independently, using

appropriate distributed technologies. By encapsulating a single business function, each microservice is adaptable across various implementations. However, adopting microservices often requires significant re-engineering, either through rebuilding applications or migrating from one paradigm to another, which can be time-consuming and prone to errors [3].

#### B. FROM MONOLITHIC TO MICROSERVICES

The complexity inherent in monolithic architectures, where components are tightly interwoven, presents substantial challenges in terms of both maintainability and scalability. As software systems grow, these issues intensify, limiting the flexibility needed for efficient development and deployment. In contrast, microservices offer a promising alternative by breaking systems into independent, cohesive units, allowing for modular design and simplified integration. However, the shift to a microservices-based approach is not straightforward; it requires careful planning, including precise migration strategies, advanced techniques for extracting microservices, and thorough assessments of service quality [5].

A key aspect of this transition is the accurate identification of microservices, which we systematically reviewed in the context of monolith decomposition [6]. The effectiveness of the new architecture largely depends on selecting services with the right level of granularity, ensuring they support agile development, are easy to maintain, and can be reused efficiently. This process requires a systematic breakdown of the existing system into distinct functional components,

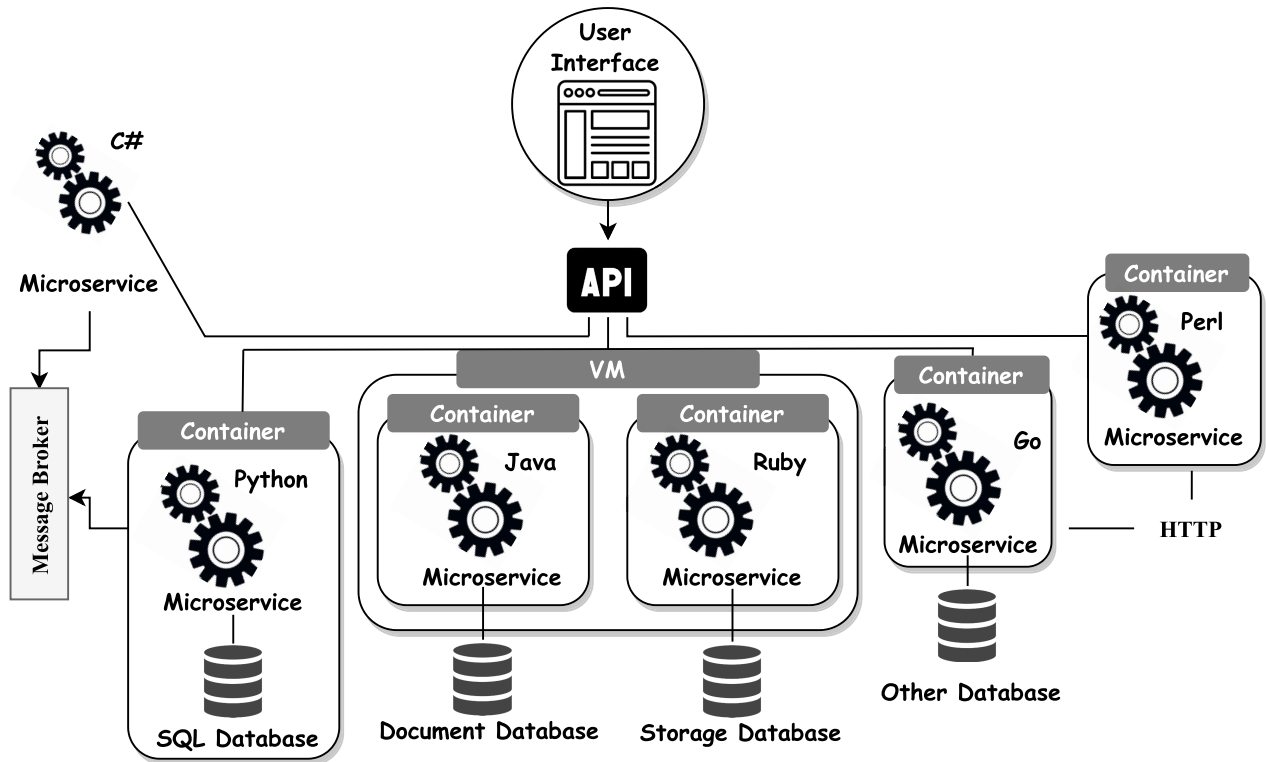


FIGURE 1. Microservices architecture.

careful delineation of service boundaries, and a combination of static and dynamic analysis methods [6], [16]. Identifying the optimal microservices is complex and often benefits from automated techniques to improve the accuracy and efficiency of the identification process [5].

### C. CATEGORIZATION

The choice of an appropriate algorithm is paramount for designing an effective activity clustering system. In this context, we employed the K-means [24], K-medoids [25], and DBSCAN [26] algorithms, each chosen for their distinctive clustering strategies suited to different data characteristics.

Given a corpus of  $N$  activities, these algorithms generate clusters (microservices) represented as  $C = \{C_1, C_2, C_3, \dots, C_K\}$ , where  $K$  is the number of clusters. K-means and K-medoids algorithms require the pre-specification of  $K$  and involve an iterative process. They initialize by establishing  $K$  initial centers (centroids for K-means and medoids for K-medoids) and assign each activity to the nearest center. This assignment continues until convergence is reached, typically determined by a minimum categorization error threshold or an unsupervised evaluation measure.

- *K-means* partitions the data by minimizing variance within each cluster, efficiently handling large datasets. However, it may struggle with non-spherical clusters and requires prior knowledge of the number of clusters [24].

- *K-medoids* selects actual data points as medoids, minimizing the total distance to all other points in the cluster. Like K-means, it assigns points iteratively, updating medoids to optimize intra-cluster distances [25].
- *DBSCAN (Density-Based Spatial Clustering of Applications with Noise)* identifies clusters based on density, grouping closely packed points while marking low-density points as outliers. This method excels at discovering clusters of arbitrary shape and is robust to noise. DBSCAN does not require specifying  $K$  in advance but instead relies on two parameters: the radius of the neighborhood around a point and the minimum number of points required to form a dense region [26].

### IV. BUSINESS PROCESSES TO MICROSERVICES

The challenge of identifying microservices involves grouping activities based on their semantic similarity to achieve low coupling and high cohesion. This approach builds upon existing decomposition techniques in the literature by incorporating advanced natural language processing and offering a more adaptable clustering framework. This approach begins by gathering detailed information about each activity, including names and descriptions, with the support of BP engineers and BP specifications to contextualize the business processes. Next, a dependency assessment is conducted, applying lexical, syntactic, and semantic analyses through NLP operations such as BIO tagging and NER to

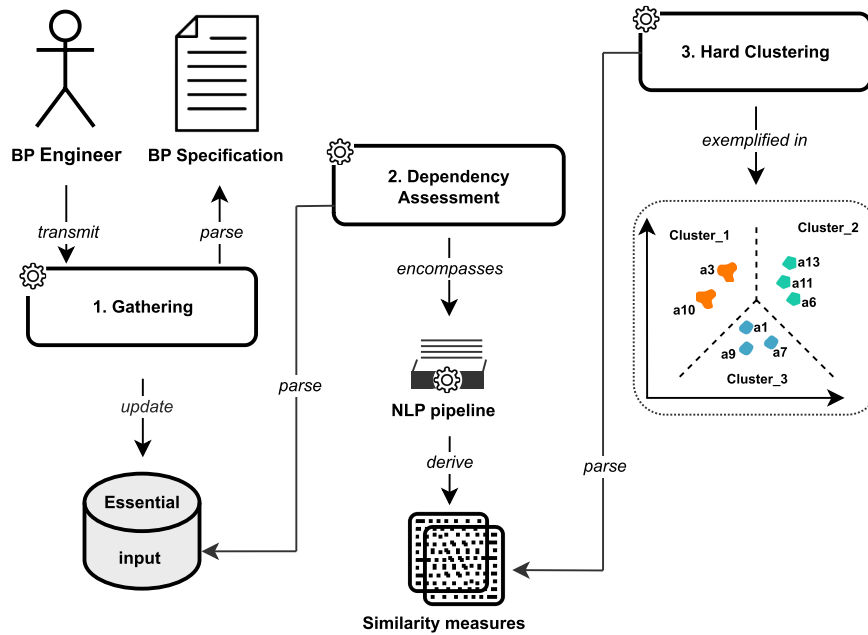


FIGURE 2. Overarching illustration of the strategy for identifying microservices.

quantify interdependencies among activities. Unlike traditional dependency analysis methods that rely primarily on structural relationships, our approach emphasizes semantic context to capture functional relationships that might not be evident from structure alone. This refined dependency information then serves as input for the hard clustering stage, where unsupervised algorithms like K-means, K-medoids, and DBSCAN organize activities into potential microservices. Our methodology differs significantly from conventional approaches in that it does not require a pre-defined number of microservices as input, addressing a key limitation in existing literature. Unlike traditional methods with predefined numbers of microservices, this approach iteratively refines clustering, supporting more accurate and adaptable microservice identification by aligning clusters with business process needs while achieving low coupling and high cohesion.

Figure 2 illustrates this process through a flowchart. Starting with BP engineers who provide BP specifications, the process moves through a three-stage pipeline. In the first stage, specifications are parsed, and essential inputs are gathered into a database, followed by NLP processing to extract data. Dependency assessments further refine the data before it undergoes hard clustering. The clustering outcome, represented by a scatter plot, shows categorized data points organized into distinct clusters, each representing a potential microservice.

#### A. DISTANCE METRIC

The categorization algorithms rely heavily on distance metrics. The selection of an appropriate distance metric is

a critical decision that impacts the entire clustering process and differentiates our approach from others in the literature. Selecting the appropriate distance metric is essential for accurately capturing the intended semantics. The distance metric, referred to as  $Dis$ , is a real-valued function for a pair of activities  $(a, a')$  that quantifies the level of difference or dissimilarity between  $a$  and  $a'$ . It adheres to the following properties  $\forall a, a', a'' \in A$ :

- 1)  $Dis(a, a') \geq 0$  (non-negativity)
- 2)  $Dis(a, a') = 0 \iff a = a'$  (identity of the indiscernible)
- 3)  $Dis(a, a') = Dis(a', a)$  (symmetrical)
- 4)  $Dis(a, a'') \leq Dis(a, a') + Dis(a', a'')$  (triangular inequality)

Normalisation of the metric is possible ( $\forall (a, a') \in A \times A; 0 \leq Dis(a, a') \leq 1$ ). In this case, the relationship between similarity  $Sim$  and the dissimilarity  $Dis$  is defined as:

$$Sim(a, a') = 1 - Dis(a, a') \quad (1)$$

After evaluating several distance metrics commonly used in existing literature, including Euclidean distance, Jaccard similarity, and Manhattan distance, we determined that cosine similarity offers superior performance for our specific context of business process activities. We have opted to use cosine similarity as the metric for evaluating semantic dependencies among activities. This measure is widely utilized in natural language processing and is effective for identifying semantic similarities within textual data [27]. It is computed as the cosine of the angle between vectors  $\vec{a}$  and  $\vec{a}'$  according to the

```

<Activity>
  <id>Task_2</id>
  <name>Deliver Bike at Anchor Point</name>
  <documentation>
    The system registers the bike's delivery at a specified anchor point location,
    marking it as available for future rentals.
  </documentation>
  <processId>BP_002</processId>
  <version>1.0</version>
  <timestamp>2024-09-26T10:15:30Z</timestamp>
</Activity>

```

**Listing 1. Activity: “Deliver Bike at Anchor Point”.**

formula:

$$Sim_{cos}(a, a') = \frac{\vec{a} \vec{a'}}{\|\vec{a}\| \|\vec{a'}\|} \quad (2)$$

Since the cosine function produces values between  $-1$  and  $1$ , the corresponding cosine distance  $Dis_{cos}$  is defined as:

$$Dis_{cos}(a, a') = 1 - Sim_{cos}(a, a') \quad (3)$$

This formulation provides two key advantages over alternative distance metrics: it normalizes for text length, making it ideal for comparing activities with varying description lengths, and it effectively captures semantic relationships rather than just lexical similarity. The formulation ensures that the distance metric for clustering remains within the range of  $0$  to  $2$ , effectively distinguishing between semantically similar and dissimilar activities. By employing cosine similarity, we can cluster semantically related business process activities, which align with potential microservices.

## B. COMPUTE SEMANTIC SIMILARITY BETWEEN ACTIVITIES

Our methodology introduces a novel multi-level NLP processing pipeline that addresses limitations in existing approaches that typically focus on only one or two linguistic levels. To effectively support microservice decomposition, the methodology leverages NLP techniques that operate at lexical, syntactic, and semantic levels to process the consolidated text of business process activities. By isolating nouns that signify core entities and removing non-essential elements like verbs, articles, adjectives, and prepositions, the approach captures valuable information necessary for structuring cohesive microservices.

Our comprehensive NLP pipeline processes business process descriptions across three distinct levels, each providing unique insights that existing approaches often overlook:

- **Lexical (or Morphological) level:** Words are decomposed into their smallest components, such as prefixes, suffixes, and root forms (morphemes). Unlike approaches that treat words as atomic units, our methodology recognizes morphological variations to identify conceptual similarities that might otherwise be missed. Methods like tokenization and lemmatization

are employed to break text into individual tokens and identify their root forms, aiding in vocabulary comprehension and text structure analysis.

- **Syntactic level:** Syntactic analysis focuses on examining the grammatical structure of sentences. This level adds critical context that purely lexical approaches in existing literature fail to capture, particularly important for business processes where action-object relationships define functionality. At this stage, Part-of-Speech (POS) tagging is applied, assigning each token a grammatical role (e.g., noun, verb, adjective) within the phrase.
- **Semantic level:** The focus is on deriving the meaning of the text, converting natural language into structured representations. We employ state-of-the-art contextual word embeddings rather than the traditional bag-of-words model seen in earlier works, enabling our approach to recognize semantic similarities even when vocabulary differs completely. Word embedding techniques [28] play a key role here, generating vector-based representations of words that capture their contextual meanings and support deeper text interpretation.

To illustrate this methodology, consider the example structure of a specific business process activity represented in XML format in Listing 1.

This structure is generated from the BP model transformed into XML format via Camunda,<sup>2</sup> maintaining critical metadata for every activity, gateway, and flow. Our approach preserves all contextual information from the original business process model, addressing a common limitation in existing methodologies that lose implementation details during abstraction. The XML format includes attributes such as activity ID, name, description, and supplementary metadata like business process ID, version, and timestamps. The name and description of each activity are merged as follows:

- **Text input:** “*Deliver Bike at Anchor Point. The system registers the bike's delivery at a specified anchor point location, marking it as available for future rentals.*”

<sup>2</sup><https://docs.camunda.org/manual/7.21/>

The combined text is then tokenized, breaking it into individual words or tokens. For instance, the tokenized version of the combined text appears as follows:

- **Tokenization:** [*“Deliver”, “Bike”, “at”, “Anchor”, “Point”, “.”, “The”, “system”, “registers”, “the”, “bike’s”, “delivery”, “at”, “a”, “specified”, “anchor”, “point”, “location”, “,”, “marking”, “it”, “as”, “available”, “for”, “future”, “rentals”, “.”*]

Next, lemmatization and stemming are applied to reduce words to their root forms, minimizing variations (e.g., “registers” becomes “register” and “delivery” becomes “deliver”). This normalization step is crucial for identifying semantic relationships that existing approaches might miss due to morphological variations. This normalization step reduces redundancy in the data and aids in recognizing that terms with slight differences are related:

- **Lemmatization and Stemming:** [*“deliver”, “bike”, “anchor”, “point”, “system”, “register”, “bike”, “deliver”, “specify”, “anchor”, “point”, “location”, “mark”, “available”, “future”, “rental”*]

Our approach then applies a domain-specific filtering process that preserves business-critical entities while reducing noise, distinguishing it from general-purpose NLP approaches in the literature. Non-essential parts of speech, such as verbs, articles, adjectives, and prepositions, are removed to focus on the most critical entities within the activity. The resulting text is distilled to its essential nouns and meaningful terms:

- **Resulting text:** [*“bike”, “anchor”, “point”, “system”, “bike”, “location”, “rental”*]

Key entities within the processed text are identified, counted, and summarized to capture the main elements of the activity. This frequency-based entity extraction provides quantitative weighting for semantic analysis, addressing limitations in existing qualitative approaches. Frequent occurrences indicate core elements that are central to the activity.

- **Key Entities:** [*bike(2), anchor(1), point(1), system(1), location(1), rental(1)*]

The final step involves creating a simplified summary of the text based on key entities. This summarization process creates a standardized semantic representation that facilitates more accurate similarity comparisons than using raw text, a significant improvement over existing approaches. This step does not add new words; it instead forms a concise version using only the main entities. So, rather than introducing new concepts, the summary reflects the primary meaning using words directly from the processed text:

- **Simplified text:** *“Bike delivery and rental at anchor point.”*

Our methodology extends beyond basic entity extraction by incorporating advanced sequence labeling techniques from the NLP domain that have not been previously applied to microservice identification. Activity annotation further

enhances this process, using BIO (Beginning-Inside-Outside) tagging and NER to segment and annotate activities within the business process text [29]. BIO tags mark the boundaries of identified entities, distinguishing whether a word starts (B), is inside (I), or outside (O) an entity. This helps in structuring phrases and isolating meaningful terms.

- **BIO labeling:**

|          |   |
|----------|---|
| Bike     | B |
| delivery | I |
| and      | O |
| rental   | I |
| at       | O |
| anchor   | B |
| point    | I |

The final NER tagging step adds domain-specific semantic classification that existing approaches typically lack, enriching the representation with business context crucial for accurate microservice identification. NER tagging is the final step, where meaningful entity types are assigned to the tagged phrases. In this case, “Bike delivery” and “rental” is recognized as an object entity because it describes an item and its associated actions, while “anchor point” is labeled as a location entity. This gives semantic structure to the information in the text. Final tagged sentence with POS, BIO, and NER:

- **Final tagged sentence with POS, BIO, and NER:**

|                 |            |
|-----------------|------------|
| Bike/Noun       | B-Object   |
| delivery/Noun   | I-Object   |
| and/Conjunction | O          |
| rental/Noun     | I-Object   |
| at/Preposition  | O          |
| anchor/Noun     | B-Location |
| point/Noun      | I-Location |

The resulting multi-dimensional representation captures functional, contextual, and semantic dimensions of business activities that are essential for accurate microservice identification but are often overlooked in existing approaches. The structured representation of each activity, defined by its core entities, serves as a basis for encoding the simplified texts into vector form. We calculate the cosine similarity between these vectors to measure semantic closeness, clustering activities that share similar contexts into cohesive microservices. This process clusters based on both entity significance and relational context, providing an adaptable framework for grouping related activities.

### C. GROUPING ACTIVITIES BY SEMANTIC SIMILARITY

Our approach to activity grouping distinguishes itself from existing literature by simultaneously optimizing for both semantic cohesion and functional coupling, rather than treating them as separate concerns. Business process activities are organized into candidate microservices by analyzing their semantic similarity. The aim is to cluster activities that contribute to similar business functionalities, ensuring they are grouped into cohesive microservices. Two critical stages

are involved in this process: evaluating semantic similarity and clustering the activities with the highest similarity together. Cohesion of microservices is enhanced, and the overall organization of the business processes is improved.

Unlike deterministic decomposition approaches in existing literature, our methodology employs a probabilistic clustering strategy specifically calibrated for business process semantics. The grouping process employs an unsupervised clustering strategy specifically designed for business process activities. Initially, the semantic similarity between each pair of activities  $A_i$  and  $A_j$  is calculated using cosine similarity, forming a semantic similarity matrix. This matrix captures the relationships between all pairs of activities and serves as the input for the subsequent clustering algorithms. The resulting similarity matrix provides a comprehensive view of the semantic landscape of business processes, enabling more informed clustering decisions than approaches that consider only pairwise relationships. The algorithm includes key input parameters that influence the grouping process.

#### • Algorithm Parameters

- *Semantic Similarity Threshold*: This represents the minimum similarity score required for activities to be clustered together, with a default value of 0.85. This threshold was determined through empirical validation across multiple business domains, offering greater adaptability than fixed thresholds used in existing approaches. Adjusting this threshold affects the granularity of the resulting clusters.
- *Coupling Threshold*: This defines the minimum coupling distance acceptable for grouping activities, with an initial value of 0.5. Higher values lead to more microservices, while lower values result in larger clusters.
- *Clustering Algorithm*: Depending on the characteristics of the business processes and the desired approach, various unsupervised clustering algorithms can be employed, including K-Means, K-Medoids, and DBSCAN. The selection of an appropriate algorithm is crucial for optimizing the clustering process.
- *Language*: The business process activities must be in English, as this is the currently supported language for analysis.

The complete pseudo-code for the method is presented in Algorithm 1. The overall process is illustrated in Figure 3, where activities are grouped into candidate microservices after semantic analysis.

## V. EMPIRICAL EVALUATION

### A. SYSTEM IMPLEMENTATION

The microservice identification system, depicted in Figure 4, consists of three main modules: Preprocessing, Dependency Assessment, and Microservice Derivation. The Preprocessing Module refines the corpus for clustering by performing essential steps like annotation, preparing the input data for

deeper analysis. Using a BP designer tool “Camunda”, a Business Process model is converted to an XML process model, and sent to the Dependency Assessment Module. This module analyzes dependencies between activities ( $\langle a_i, a_j \rangle$ ) to generate a dependency graph, represented in XML format, to capture interdependencies among activities.

Next, the Microservice Derivation Module processes the XML dependencies using Java DOM.<sup>3</sup> Here, clustering techniques are applied to group related activities into candidate microservices, producing a microservice architecture as distinct clusters. The final output of this module is an XSD (XML Schema Definition), ensuring compatibility with downstream applications, along with a compiled representation of candidate microservices. Our implementation differs from existing approaches in several key aspects: it provides a fully automated pipeline from business process models to microservice candidates without requiring manual intervention at intermediate stages; it incorporates advanced NLP techniques that operate at multiple linguistic levels simultaneously; and it offers flexible parameterization of thresholds and clustering algorithms to adapt to different business domains.

### B. EVALUATION DATASETS

To demonstrate the broad applicability and effectiveness of our approach across diverse business domains, we selected three distinct case studies representing different application types and complexities. Each dataset presents unique challenges that test various aspects of our methodology.

#### • Bicing

Bicing, a bicycle-rental scheme in Barcelona, boasts a fleet of 6,000 bicycles and a network of 400 anchor stations. The monolithic design of Bicing, coupled with operational and technical challenges such as managing large development teams and ensuring software scalability, presented obstacles to the system’s optimal functioning [30]. Figure 4 11 depicts a high-level Business Process Model and Notation (BPMN) representation of key Bicing activities ( $a_i$ ), gateways, dependencies, and artifacts meticulously designed for this study.

A user initiates the rental process by requesting a bicycle ( $a_1$ ) from a designated anchor point. After confirming the user’s information ( $a_2$ ) as well as any delinquencies ( $a_3$ ), Bicing updates the user’s information ( $a_4$ ) and grants the rental request ( $a_5$ ). If the selected bicycle is unsuitable, the user may replace it ( $a_6$ ) or request a different one. Once a suitable bicycle is obtained, the user’s rental journey commences ( $a_{10}$ ). All bikes are regularly maintained ( $a_7$ ) and then either returned ( $a_8$ ) or discarded ( $a_9$ ). The renter returns the bicycle to an anchor point upon reaching the destination ( $a_{11}$ ). Improper bicycle return triggers a user ban

<sup>3</sup><https://xerces.apache.org/xerces2-j/javadocs/xerces2/org/apache/xerces/dom>

<sup>4</sup>In consideration of the substantial dimensions of the image, the illustrations have been repositioned to the final pages of the document.

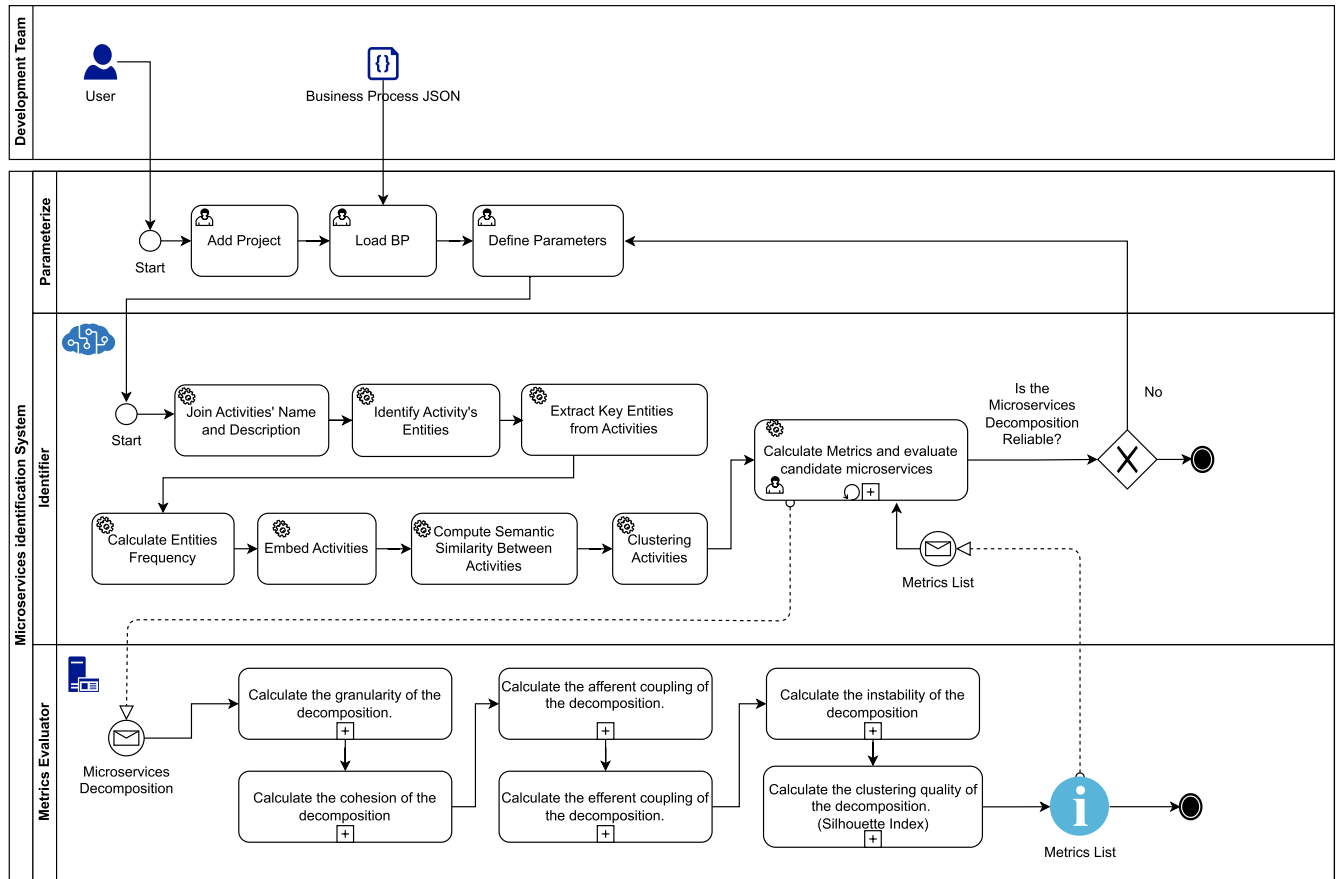


FIGURE 3. Microservice identification pipeline.

( $a_{14}$ ) and prompts Bicing to locate the bicycle ( $a_{12}$ ), enabling its recovery and subsequent availability for future rentals ( $a_{13}$ ).

#### • Cargo Tracking

In Baresi et al. [13] Cargo Tracking application is described (Figure<sup>5</sup> 12). The process commences when a shipping firm transports the containers of a client ( $a'_1$ ) over both land and sea. After verifying the client's information ( $a'_2$ ) and the viability of the shipping alternatives ( $a'_3$ ), the business either sends a payment invoice ( $a'_5$ ) or inform the client of the denial ( $a'_4$ ). Additional actions are conducted upon the confirmation of a payment. Firstly, ( $a'_7$ ) coordinates the logistics of shipping containers by land and sea, while ( $a'_8$ ) monitors the warehouses' arriving and departing movements. Secondly, ( $a'_{10}$ ) monitors whether the containers have been cleared for travel at probable port stops whilst ( $a'_9$ ) puts them aboard the ship. Once the cargo reaches its final destination, ( $a'_{11}$ ) is responsible for emptying the containers off the ship and placing them onto the trucks. Last but not least, ( $a'_{12}$ ) files infractions that are eventually declared by customs. In the event of any discrepancies, the shipper of the containers

would be obliged to pay sanctions, bringing the procedure to a conclude ( $a'_{13}$ ). In contrast, ( $a'_{14}$ ) manages the logistics of trucking shipments, whereas ( $a'_{15}$ ) monitors containers between depots prior to delivery.

#### • JPpet store

The JPpetStore Demo<sup>6</sup> is an online pet store that allows users to explore a catalog of products [15], add items to a shopping cart, modify the cart, and place orders. While many actions are available without logging in, users must create an account and sign in to complete a purchase. Key features of JPpetStore include:

- Account registration and login.
- product catalog interactions, such as browsing and searching.
- Shopping cart management: adding, removing, and adjusting item quantities, as well as reviewing and submitting orders.

JPpetStore is frequently used to test approaches for breaking down monolithic applications into microservices. A BPMN diagram illustrating the application's processes, where each

<sup>5</sup>In consideration of the substantial dimensions of the image, the illustrations have been repositioned to the final pages of the document.

<sup>6</sup><http://demo.kieker-monitoring.net/jppetstore/help.html>

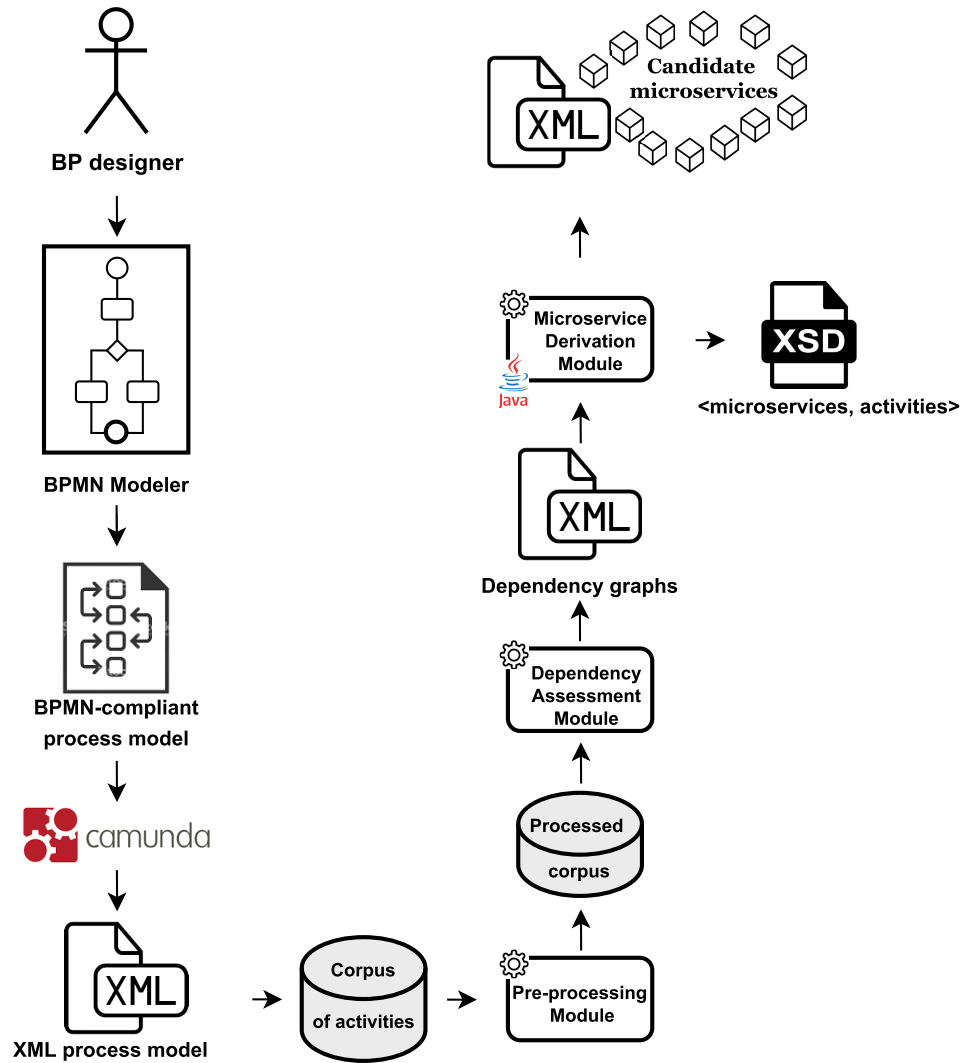


FIGURE 4. The system's design architecture.

activity can be further detailed in sub-diagrams, is shown in Figure<sup>7</sup> 13.

### C. EVALUATION METRICS

To evaluate the decompositions generated by each method, various metrics adapted from prior work [31], [32], [33] are used. These metrics cover coupling, cohesion, stability, granularity, and performance, focusing on the following five specific metrics.

#### • Granularity (N)

This metric measures the count of microservices within the system decomposition.

#### • Silhouette Index (SI)

The Silhouette Index assesses how well a point aligns with its cluster versus others, where higher scores suggest more distinct clusters, with a range from  $-1$  to  $1$ . This index

<sup>7</sup>In consideration of the substantial dimensions of the image, the illustrations have been repositioned to the final pages of the document.

helps determine the effectiveness of clustering in representing microservices, defined by:

$$SI = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (4)$$

Here,  $a(i)$  is the average distance from point  $i$  to others in its cluster, and  $b(i)$  is the average distance to the nearest neighboring cluster. The silhouette index can also help identify the optimal cluster count  $K$ , balancing cohesion and separation.

#### • Afferent Coupling (AC)

This metric tracks the number of microservices relying on a particular microservice, indicating its significance within the system. For a microservice  $M_i$ , afferent coupling is calculated as:

$$AC_i = \sum_{j=1}^n \text{dependence}(M_j, M_i) \quad (5)$$

**Algorithm 1** Microservices Identification Algorithm

---

**Input:** list of business processes  $BP$ , Semantic Similarity Threshold  $SST$ , Coupling Threshold  $CT$ , Clustering Algorithm  $CA$ , Language  $L$

**Output:** set of candidate microservices  $M$ , clustering metrics  $metrics$

Initialize an empty list  $processed\_activities$ ;

**foreach**  $activity$  in  $BP$  **do**

    Convert the  $activity$  name and description to lowercase;

    Remove punctuation and stop words from the  $activity$  content;

    Lemmatize the  $activity$  content;

    Tokenize the  $activity$  content by splitting on spaces;

    Append the processed activity to the list  $processed\_activities$ ;

**end**

Initialize an empty list  $annotated\_activities$ ;

**foreach**  $processed\_activity$  in  $processed\_activities$  **do**

    Apply POS (Part-of-Speech) tagging to label each term in the  $processed\_activity$ ;

    Identify key entities using Named Entity Recognition (NER);

    Apply BIO (Beginning, Inside, Outside) tagging to mark entity boundaries;

    Append the annotated activity to the list  $annotated\_activities$ ;

**end**

Initialize an empty similarity matrix  $similarity\_matrix$ ;

**foreach** pair of  $annotated\_activities$  **do**

    Compute the semantic similarity between pairs of activities using cosine similarity;

**if**  $similarity\_score > SST$  **then**

        Record the similarity score in the  $similarity\_matrix$ ;

**end**

**end**

Initialize an empty list  $clusters$ ;

**foreach**  $annotated\_activity$  in  $annotated\_activities$  **do**

**foreach** other  $activity$  in  $annotated\_activities$  **do**

**if**  $coupling\_score \leq CT$  **then**

            Assign both activities to the same cluster;

**end**

**end**

**end**

Apply the selected clustering algorithm  $CA$  to group activities based on similarity and coupling criteria;

Compute clustering metrics (e.g., cohesion, separation) and store them in  $metrics$ ;

**return** set of candidate microservices  $M$ , clustering metrics  $metrics$ ;

---

where  $dependence(M_j, M_i)$  is 1 if  $M_j$  depends on  $M_i$ , and 0 otherwise.

- **Efferent Coupling (EC)**

This metric counts the dependencies a microservice has on others, reflecting its complexity. For  $M_i$ , efferent coupling is given by:

$$EC_i = \sum_{j=1}^n dependence(M_i, M_j) \quad (6)$$

where  $dependence(M_i, M_j)$  is 1 if  $M_i$  depends on  $M_j$ , and 0 otherwise.

- **Instability (I)**

Instability, calculated as the ratio of efferent coupling to total coupling (afferent + efferent), measures a microservice's resilience to changes. It ranges from 0 (fully stable) to 1 (fully unstable), defined as:

$$I_i = \frac{EC_i}{AC_i + EC_i} \quad (7)$$

- **Relational Cohesion (RC)**

This metric assesses the ratio of internal connections, such as method calls and attribute access, to the total types within a microservice. For  $M_i$ , relational cohesion is calculated as:

$$RC_i = \frac{\text{Internal Relations}}{\text{Total Types}} \quad (8)$$

Higher values of  $RC$  signify greater internal coherence.

## D. EVALUATION RESULTS

The comprehensive results of employing the proposed technique on the project dataset are presented in Table 2.

For the Bicing project, the proposed approach consistently produces a granularity of four microservices across all clustering algorithms (K-means, K-medoids, DBSCAN), compared to five with the MM method [8]. In terms of dependency metrics, the approach shows lower **AC** and **EC** with K-means ( $AC = 15.5$ ,  $EC = 0.5$ ) than MM ( $AC = 17.6$ ,  $EC = 0.8$ ), suggesting potentially lower interdependencies in this configuration. Instability remains low across all methods, with DBSCAN showing a slightly higher instability at **0.08**, while K-means and K-medoids achieve stability levels similar to MM.  $RC$  varies slightly, with K-means and DBSCAN achieving values of **4.5** and **4.0**, respectively, indicating a reasonable level of cohesion in decomposition.

In practical terms, the microservices identified for Bicing using the K-means approach would represent a more maintainable architecture with clearer boundaries and fewer cross-service dependencies, facilitating independent deployment and scaling—critical requirements for urban mobility applications that experience variable load patterns throughout the day.

In the Cargo Tracking project, the clustering algorithms in the proposed approach yield varied results. K-means achieves **AC = 13.0** and **EC = 3.0**, showing a balanced reduction in coupling compared to methods like Service Cutter ( $AC = 13.5$ ,  $EC = 3.3$ ) [22]. Service Cutter performs similarly in terms of **AC** and **EC** but does not achieve the same level of stability as K-medoids (**I = 0.15**), with Service Cutter exhibiting slightly higher instability. Furthermore, Service Cutter does not show as much improvement in **RC** when compared to the proposed approach. In contrast, the DFD method [20] also provides a high **RC** value, but it does not achieve as favorable results in terms of coupling and instability metrics as K-medoids or K-means. The Instability metric for DFD is slightly higher, indicating a somewhat less stable microservice decomposition.

For the logistics domain represented by Cargo Tracking, the K-medoids configuration delivers particularly compelling results with the lowest instability ( $I = 0.15$ ) and highest cohesion ( $RC = 13.5$ ) among all methods. This configuration would translate to a microservice architecture where changes to one service are less likely to propagate to others—a crucial characteristic for supply chain systems that must evolve different components at different rates due to varying regulatory and business requirements.

**TABLE 2.** Results for evaluation projects.

| Project        | Method         | N | AC    | EC   | I    | RC    |                                   |
|----------------|----------------|---|-------|------|------|-------|-----------------------------------|
| Bicing         | MM             | 5 | 17.6  | 0.8  | 0.05 | 5     |                                   |
|                |                | 4 | 15.5  | 0.5  | 0.15 | 4.5   |                                   |
|                | Our approach   | 4 | 16.25 | 3.25 | 0.04 | 3.75  | <i>K-means</i>                    |
|                |                | 4 | 15.0  | 2.5  | 0.08 | 4.0   | <i>K-medoids</i><br><i>DBSCAN</i> |
| Cargo Tracking | MM             | 5 | 15.2  | 3.6  | 0.21 | 13.25 |                                   |
|                | Genetic        | 3 | 14.0  | 4.0  | 0.18 | 12.0  |                                   |
|                | DFD            | 3 | 13.3  | 2.7  | 0.2  | 14.2  |                                   |
|                | Service cutter | 4 | 13.5  | 3.3  | 0.2  | 12.2  |                                   |
|                |                | 4 | 13.0  | 3.0  | 0.19 | 13.0  |                                   |
|                | Our approach   | 4 | 12.0  | 2.5  | 0.15 | 13.5  | <i>K-means</i>                    |
|                |                | 4 | 12.5  | 2.8  | 0.16 | 13.2  | <i>K-medoids</i><br><i>DBSCAN</i> |
| Jpet Store     | Genetic        | 5 | 12.5  | 3.0  | 0.25 | 10.5  |                                   |
|                |                | 4 | 14.5  | 2.5  | 0.12 | 11.0  |                                   |
|                | Our approach   | 4 | 14.0  | 2.8  | 0.11 | 11.5  | <i>K-means</i>                    |
|                |                | 4 | 13.5  | 3.2  | 0.13 | 10.8  | <i>K-medoids</i><br><i>DBSCAN</i> |

**MM** : Multi-Model (Daoud, M., et al., 2021)

**Genetic** : Genetic algorithm of microservice backlog (Vera-Rivera et al., 2021).

**Service cutter** : (Gysel et al., 2016)

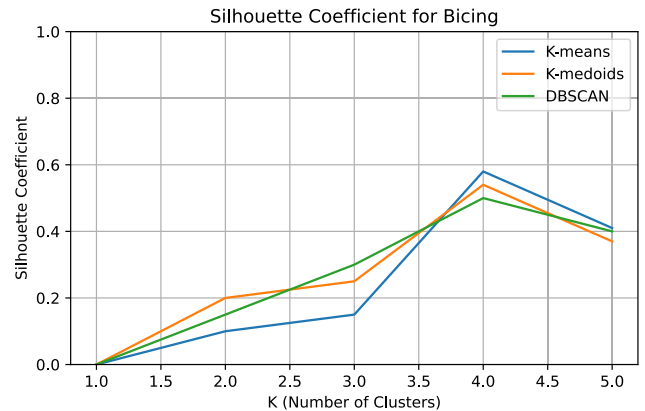
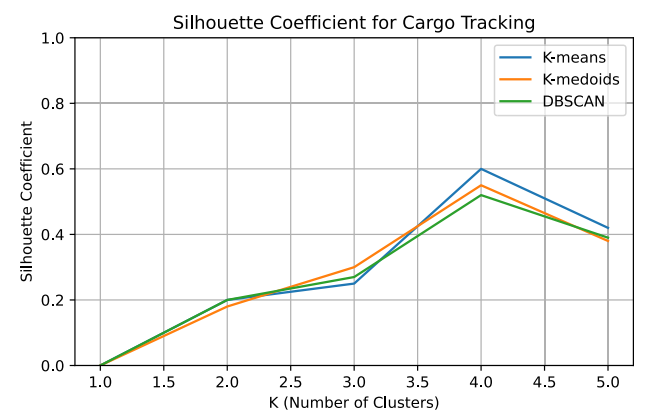
**DFD** : Data Flow Diagram (Li, S., et al., 2019).

For the JPetStore project, the approach yields a comparable granularity of four microservices across clustering algorithms, while the study by Vera-Rivera et al. [12] using a Genetic method results in five microservices for the same system. The proposed approach shows relatively balanced AC and EC values, with K-means achieving **AC = 14.5** and **EC = 2.5**, indicating similar dependencies to other methods. Instability scores remain low, with K-means at 0.12 and K-medoids at **0.11**, suggesting stability across configurations. Relational Cohesion is also favorable, with **RC = 11.5** achieved with K-medoids, reflecting effective clustering with cohesive internal relations.

From a practical implementation perspective, the microservice decomposition identified for JPetStore using the K-medoids approach offers an optimal balance between modularity and operational complexity. With four well-defined services instead of five, development teams would manage fewer deployment pipelines and service interactions while maintaining strong functional cohesion, particularly beneficial for e-commerce platforms where feature development velocity and system stability are equally important.

The comparison of the K-means, K-medoids, and DBSCAN algorithms based on their average silhouette coefficient for Bicing is shown in Figure 5. The silhouette scores show that K-means achieves the highest silhouette coefficient, indicating well-defined clusters and clear activity grouping for potential microservices. K-medoids and DBSCAN also perform similarly, though with slightly lower silhouette values, suggesting that all three methods provide reasonably cohesive clusters for this project.

In Figure 6 for the Cargo Tracking project, K-means again achieves the highest silhouette coefficient, followed closely by K-medoids. DBSCAN demonstrates slightly lower cohesion, though still within an acceptable range for activity grouping. These results suggest that K-means and

**FIGURE 5.** Silhouette Coefficient - Bicing case.**FIGURE 6.** Silhouette Coefficient - Cargo case.

K-medoids provide effective clustering for this case study, achieving well-balanced clusters that support microservice identification.

For the JPetStore project in Figure 7, K-means and K-medoids yield similar silhouette coefficients, indicating

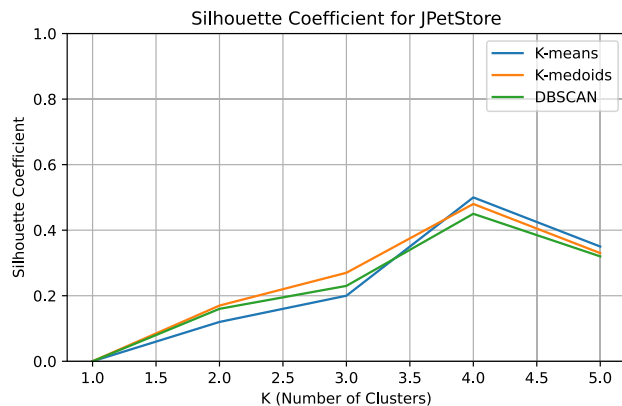


FIGURE 7. Silhouette Coefficient - JPetStore case.

strong internal consistency within the clusters. DBSCAN produces a lower silhouette score compared to the other algorithms, though it still maintains reasonably defined clusters for the purpose of identifying microservices. Overall, K-means and K-medoids remain effective for this project, providing more cohesive clustering outcomes.

Figure 8 presents a comprehensive comparison of various clustering methods—K-means, K-medoids, and DBSCAN—across multiple metrics for the Bicing, Cargo Tracking, and JPet Store projects. Starting with granularity, the plot shows that your approach consistently identifies a different number of microservices compared to the other methods. K-means and K-medoids, for instance, tend to identify a similar number of microservices across the three projects, suggesting that these methods provide a more granular identification of microservices. In comparison, methods like DBSCAN and MM [8] identify fewer microservices in the Bicing and JPet Store projects, but DBSCAN shows a slightly higher count in the Cargo Tracking project. This indicates that DBSCAN, due to its density-based nature, is more conservative in its microservices identification, grouping them into fewer but larger clusters. Genetic [12] and DFD [20] approaches, on the other hand, seem to focus more on specific features of the system, potentially influencing the number of microservices detected differently from the unsupervised methods.

When examining Afferent Coupling, it becomes evident that K-means and K-medoids generate microservices with higher AC values across the projects, suggesting a more interconnected structure. In the Cargo Tracking project, K-means yields the highest AC values, indicating that the identified microservices are more dependent on others, reflecting a tightly coupled service structure. Genetic [12] and Service Cutter [22] methods, however, demonstrate a lower interconnection between their identified microservices, which could imply that these methods focus on creating more independent services. MM [8] also shows varied results in AC, with certain projects exhibiting higher interdependencies, but in general, its performance tends to fall between K-means/K-medoids and DBSCAN.

The Efferent Coupling chart, reveals that DBSCAN consistently generates microservices with lower EC values compared to K-means and K-medoids. This suggests that DBSCAN produces more isolated microservices, which could be advantageous in specific scenarios requiring self-contained units. Service Cutter [22] and MM [8], depending on the configuration, also lean towards producing microservices with lower EC values, though their focus on design principles and service boundaries varies. Genetic method [12], while showing a balance in EC, generally produce microservices that are more interconnected than DBSCAN but slightly less so than K-means and K-medoids.

Finally, when considering Instability, which measures how likely microservices are to change based on their dependencies, the plot shows that DBSCAN tends to produce microservices with higher instability, especially in the Cargo Tracking project. This suggests that DBSCAN's clustering results are more sensitive to variations in the data. Conversely, K-means and K-medoids exhibit lower instability values, particularly in the Bicing and JPet Store projects, indicating that the microservices identified by these methods are more stable and less prone to change. The Genetic approach [12], while typically more stable in some cases, can show higher instability depending on the complexity of the system. Service Cutter [22], focusing on predefined service boundaries, often results in lower instability by ensuring a more rigid structure for each service. MM [8], depending on the model used, shows varying instability across projects, but generally falls between DBSCAN and K-means/K-medoids.

The stacked bar chart for Dependencies ( $AC + EC$ ) in Figure 9 visually represents the Afferent Coupling and Efferent Coupling values across different clustering methods for the projects. For instance, methods like K-means and K-medoids tend to show a more balanced distribution between AC and EC, suggesting that these methods result in microservices with relatively even incoming and outgoing dependencies. DBSCAN, however, exhibits more variation across the projects, with Cargo Tracking in particular showing higher AC and lower EC values. This indicates that DBSCAN may group microservices in a way that leads to more centralized dependencies (higher AC), which could differ from the more balanced dependency distribution seen with K-means and K-medoids.

When comparing the MM method [8] to other methods like DFD [20], Service Cutter [22], and Genetic [12], it appears that MM tends to produce microservices with more balanced AC and EC values across the different projects. In contrast, methods such as DFD and Service Cutter may focus more on predefined dependency relationships, which can lead to microservice structures that are less flexible and potentially more rigid. The Genetic algorithm, while adaptive, focuses on optimizing the overall service architecture and may sometimes result in higher AC values, but with less consistency in dependency distribution compared to MM.

Moving to the heatmap for correlation between metrics, Figure 10 highlights a positive correlation between AC

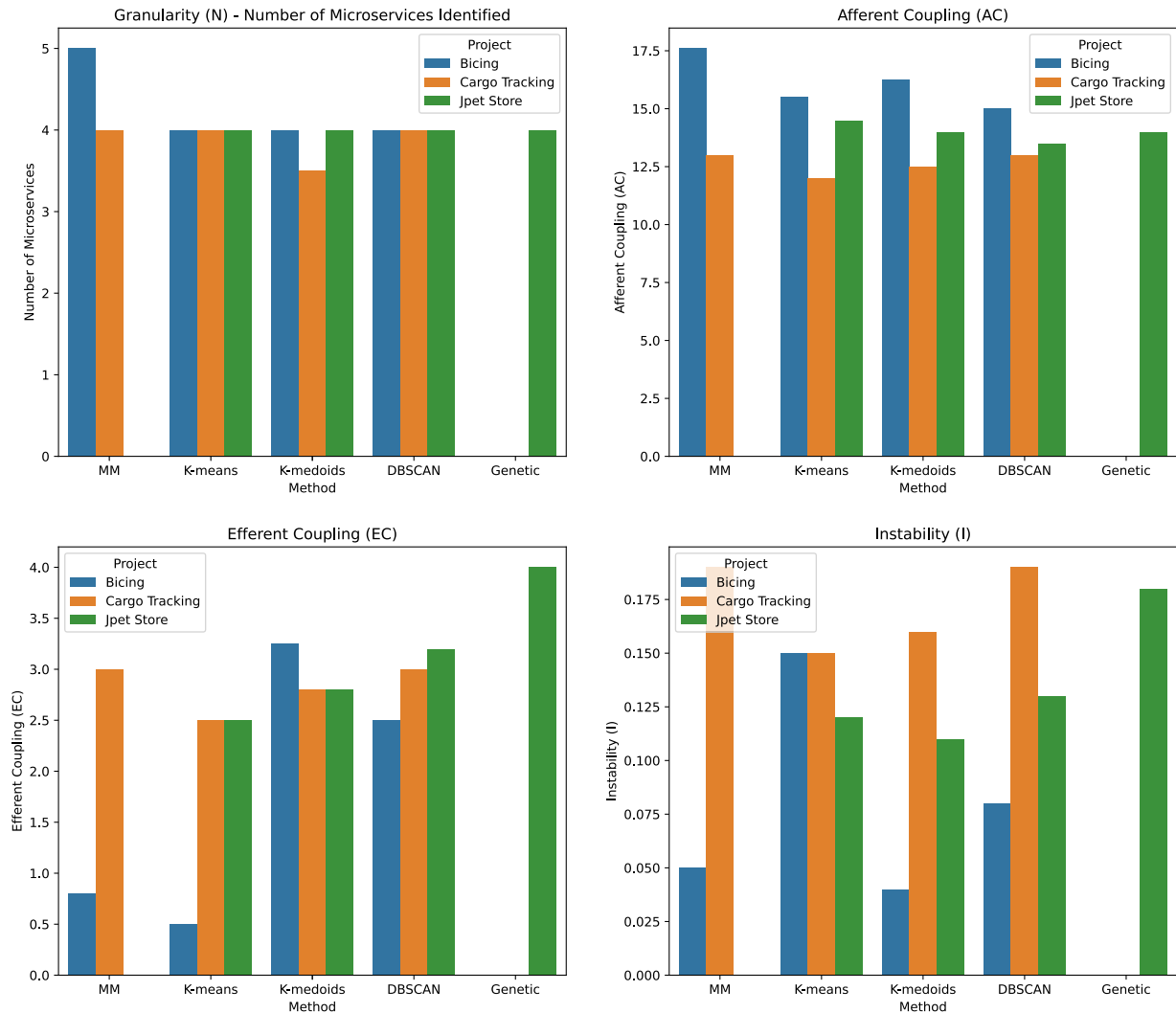


FIGURE 8. Graphical results of the metrics.

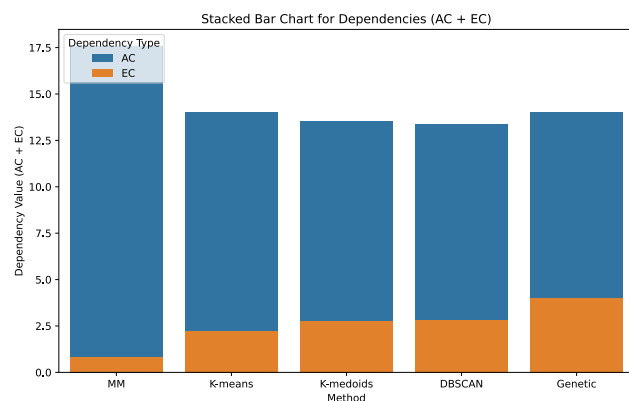


FIGURE 9. Stacked bar chart for dependencies (AC + EC).

and *EC*, suggesting that higher interdependencies between microservices tend to coincide with a more cohesive structure. Similarly, the correlation between *I* and *RC* shows that

microservices with greater interdependencies are often more unstable but also tend to exhibit stronger internal cohesion.

This insight helps to understand how different methods affect the interconnectedness and stability of the identified microservices. For example, **MM** achieves a generally balanced distribution of dependencies, which may contribute to more stable microservices. In comparison, methods like **DBSCAN** might lead to microservices with more centralized dependencies, which could influence their stability and cohesion differently. These correlations thus provide a better understanding of the impact of clustering methods on the overall architecture, offering guidance for selecting the most appropriate approach based on the desired balance between cohesion, stability, and dependency distribution.

The evaluation results demonstrate several key advantages of the approach that highlight its practical applicability. First, the semantic-driven approach consistently produces microservice decompositions with lower coupling (both AC

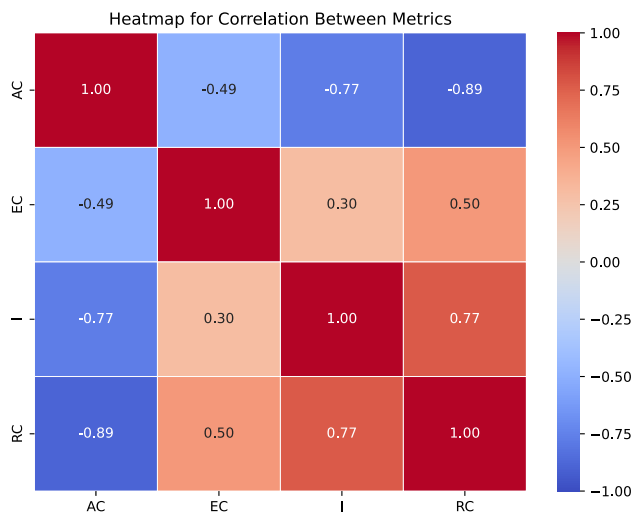


FIGURE 10. Correlation between metrics.

and EC) compared to existing methods across all three case studies. This translates directly to practical benefits in enterprise settings, where lower coupling facilitates independent development, testing, and deployment of microservices by separate teams.

Second, the stability metrics show that the approach creates more resilient microservice architectures that are less susceptible to cascading changes. This has significant practical implications for system maintenance and evolution, as changes to one microservice are less likely to propagate unexpectedly to others.

Third, the consistent silhouette coefficient values across different clustering algorithms demonstrate the robustness of the approach regardless of the specific clustering technique employed. This flexibility allows practitioners to select the clustering algorithm that best fits their organizational constraints without compromising the quality of the resulting microservice architecture.

The applicability of the approach extends beyond the evaluation cases presented here. Organizations can integrate this methodology into their existing DevOps pipelines by:

- Using the XML output from our system as input for automated code generation tools that scaffold microservice implementations based on the identified boundaries.
- Incorporating the dependency information into architectural documentation and service contracts to guide development teams.
- Leveraging the instability and cohesion metrics to prioritize refactoring efforts in legacy system modernization.
- Utilizing the clustering visualization capabilities to facilitate discussions between business and technical stakeholders about service boundaries.

Future extensions of this work could include domain-specific language adapters to handle multilingual business process specifications, integration with runtime monitoring

tools to validate and refine microservice boundaries based on actual system behavior, and the development of specialized clustering algorithms optimized specifically for business process semantics rather than relying on general-purpose clustering techniques.

## VI. LIMITATIONS

This approach builds on a strong foundation, extending prior work on microservices identification through innovative techniques that enhance business process comprehension and modularity preservation [7], [8]. By employing BIO-tagged Named Entity Recognition and leveraging the power of NLP, this extension offers improved accuracy in extracting and structuring microservices. To identify use cases from state-of-the-art case studies, we utilized information reported in published papers, carefully interpreting the business logic and dependencies to avoid bias [17], [18]. Nevertheless, a few considerations remain that could further refine and generalize the approach.

The approach relies on high-quality and comprehensive business process documentation. Although BIO-tagged NER and NLP tools effectively parse data from textual descriptions, the accuracy of microservices identification is influenced by the completeness and clarity of these documents. As noted in related studies, any ambiguity or inconsistency in source documentation can hinder the correct interpretation of dependencies and boundaries among services [12], [22]. Additionally, for especially large and complex business applications, manually verifying dependencies or interpreting boundary definitions can be resource-intensive. Process experts or domain architects may find it useful to provide additional validation in cases of ambiguous dependencies, especially when business logic spans across multiple domains [5], [23].

Scalability presents another key area for improvement. As the number of activities and dependencies increases, the computational load for NLP and clustering algorithms grows, potentially necessitating additional computational resources or parameter fine-tuning for larger datasets. This challenge aligns with the scalability limitations seen in other clustering models for complex systems [3]. Future optimizations could focus on adaptive clustering techniques to better support larger-scale applications.

Parameter sensitivity is also a notable factor. The clustering algorithms used in this methodology depend on similarity and coupling thresholds, which must be precisely calibrated for each application context. As observed in other clustering-based methods, small adjustments in these parameters can significantly impact clustering results [32], [34]. Future work could integrate adaptive mechanisms to dynamically adjust these parameters, potentially increasing robustness across various datasets [6].

While this approach optimizes modularity and cohesion, achieving an ideal balance between clustering boundaries and service modularity remains challenging. Subjective boundary definitions may sometimes require an experienced architect's

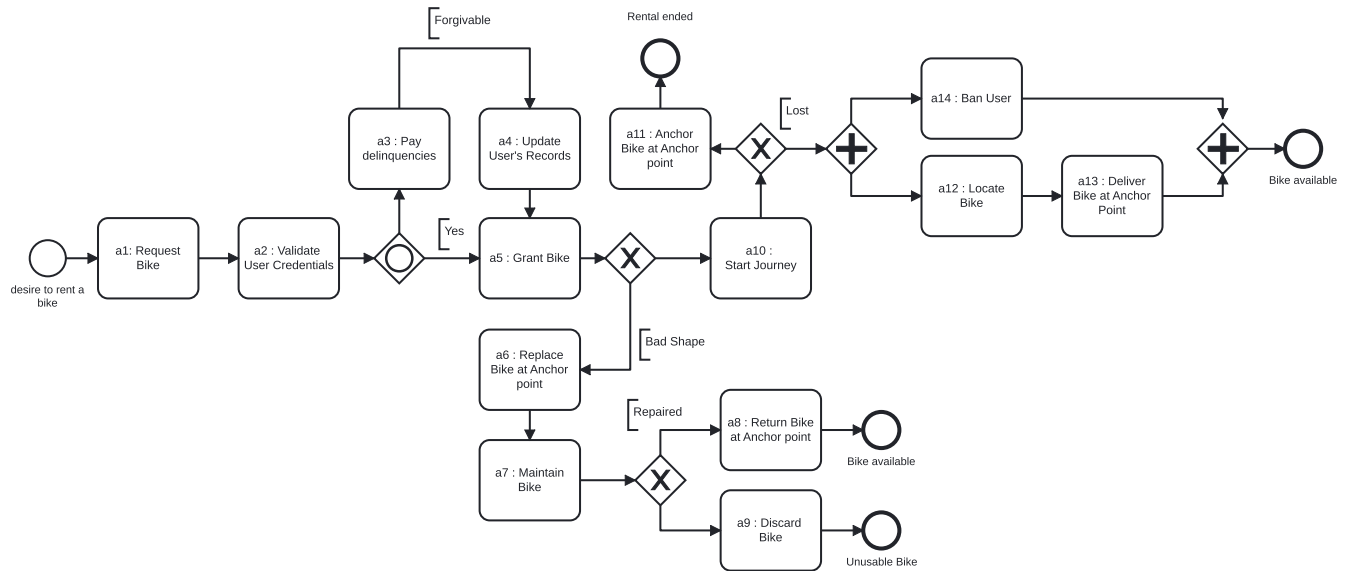


FIGURE 11. A BPMN diagram depicting Bicing.

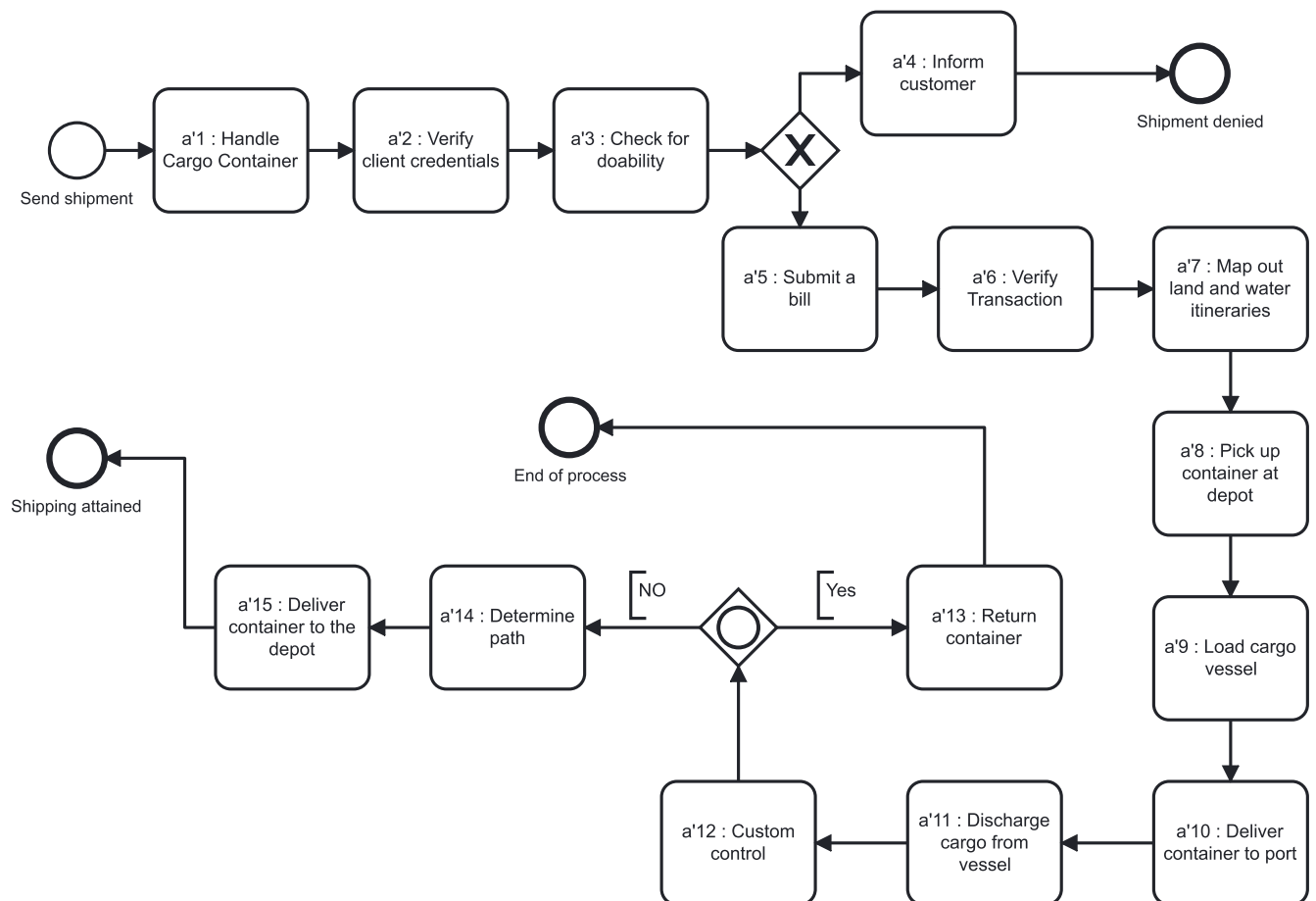
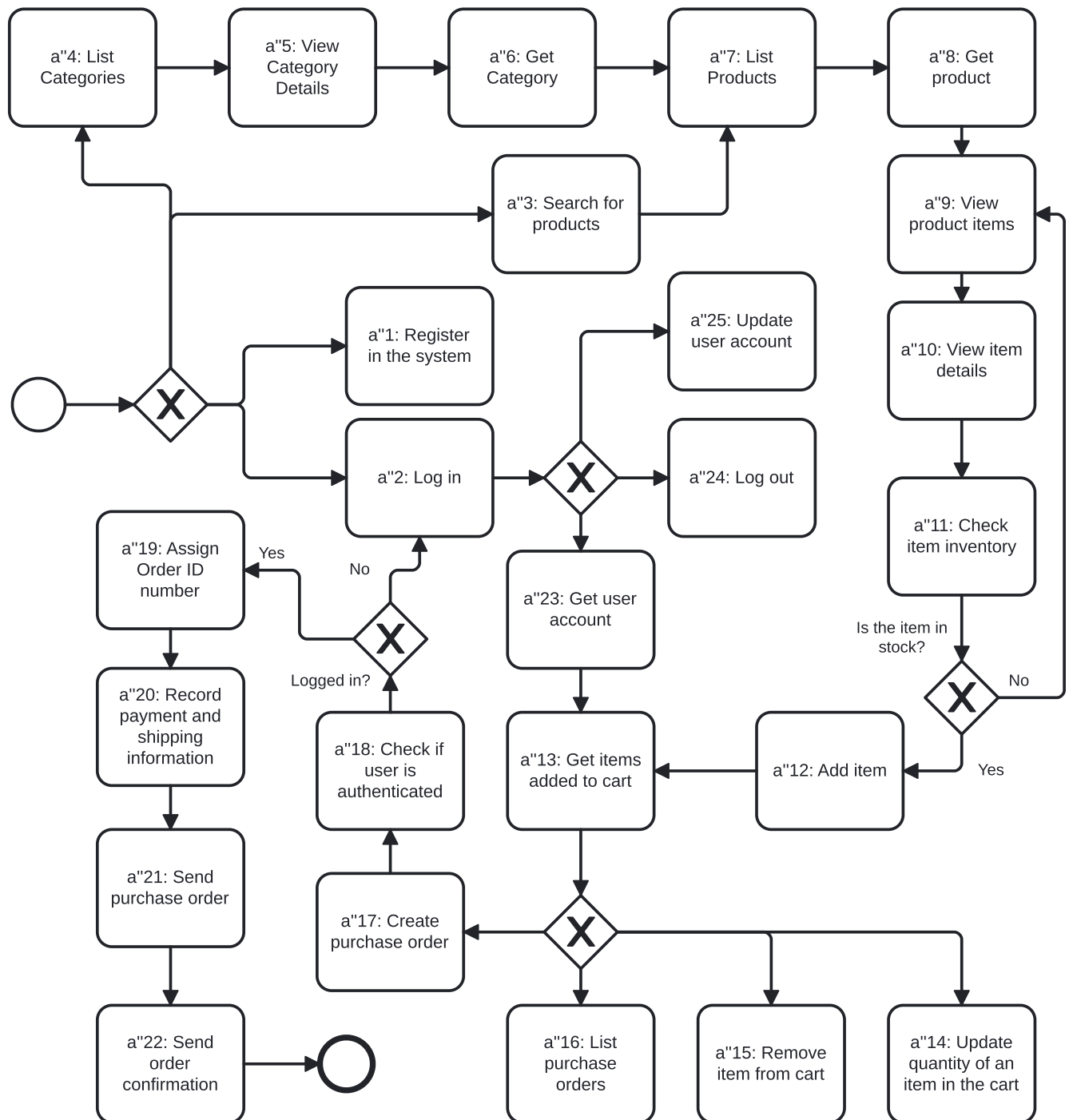


FIGURE 12. A BPMN diagram depicting Cargo Tracking.

input to refine clusters, especially when clustering results are less clear-cut [13], [22]. Lastly, this extension could explore alternative clustering algorithms or multi-objective

optimization methods. Approaches like ant colony optimization and multi-objective clustering could enhance the methodology's robustness in complex, modularity-intensive



**FIGURE 13.** A BPMN diagram depicting JPét-Store.

scenarios, offering additional resilience and flexibility across diverse business environments [3], [34].

## VII. CONCLUSION

Identification of microservices is a critical challenge for organizations transitioning to fine-grained, coherent, and loosely coupled architectures. While various methods rely

on inputs like log files, domain ontologies, UML diagrams, and databases, the approach uniquely utilizes BPs to gain a comprehensive understanding of business operations. The approach employs natural language processing to capture interdependencies across business process activities based on annotated BPs. These dependencies are then integrated using a clustering approach, providing inputs for the

K-means, K-medoids, and DBSCAN algorithms to identify potential microservices. The strategy outperforms hierarchical clustering in precision, resulting in more decoupled and cohesive microservices. The strategy outperforms hierarchical clustering in precision, resulting in more decoupled and cohesive microservices. When compared to existing approaches, our method demonstrates several advantages: it achieves higher semantic coherence by leveraging business process knowledge rather than solely relying on technical artifacts, it produces more stable microservice boundaries that better reflect actual business domains compared to code-based decomposition approaches, and it reduces the expertise required for successful implementation compared to manual identification methods. This success stems from the approach's ability to capture the rich semantic context of BPs, enabling the identification of microservices that align with business domain boundaries.

Future work will investigate the applicability of different activity dependence patterns for identifying microservices, explore the impact of security needs on microservice identification, consider customizable BPs and their specificities as input, explore alternative clustering strategies, and investigate BP's activity code to extract valuable information that complements the existing model. Additionally, future research will quantitatively evaluate our approach against other state-of-the-art methods using standardized metrics for coupling, cohesion, and business alignment to further validate the improvements observed in this study. By addressing these research directions, we aim to further refine the approach and establish it as a powerful tool for identifying microservices from BPs, enabling organizations to reap the benefits of microservice architectures with greater ease and effectiveness.

## REFERENCES

- [1] T. Davenport and J. Short, *The New Industrial Engineering: Information Technology and Business Process Redesign*. Cambridge, MA, USA: Sloan Manage. Rev., 1990.
- [2] E. Heydari Beni, B. Lagaisse, and W. Joosen, "Infracomposer: Policy-driven adaptive and reflective middleware for the cloudification of simulation & optimization workflows," *J. Syst. Archit.*, vol. 95, pp. 36–46, May 2019.
- [3] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds., Cham, Switzerland: Springer, 2017, pp. 195–216.
- [4] I. Oumoussa, S. Faieq, and R. Saidi, "Microservices: Investigating underpinning," in *Proc. 3rd Int. Conf. Digit. Age Technol. Adv. Sustain. Develop.* Cham, Switzerland: Springer, 2022, pp. 343–351.
- [5] M. Abdellatif, A. Shatnawi, H. Mili, N. Moha, G. E. Boussaidi, G. Hecht, J. Privat, and Y.-G. Guéhéneuc, "A taxonomy of service identification approaches for legacy software systems modernization," *J. Syst. Softw.*, vol. 173, Mar. 2021, Art. no. 110868.
- [6] I. Oumoussa and R. Saidi, "Evolution of microservices identification in monolith decomposition: A systematic review," *IEEE Access*, vol. 12, pp. 23389–23405, 2024.
- [7] I. Oumoussa, R. Saidi, and S. Faieq, "A business-centric approach to automated microservices identification," in *Digital Technologies and Applications* (Lecture Notes in Networks and Systems), vol. 1101, S. Motahhir and B. Bossoufi, Eds., Cham, Switzerland: Springer, 2024.
- [8] M. Daoud, A. El Mezouari, N. Faci, D. Benslimane, Z. Maamar, and A. El Fazziki, "A multi-model based microservices identification approach," *J. Syst. Archit.*, vol. 118, Sep. 2021, Art. no. 102200.
- [9] M. J. Amiri, "Object-aware identification of microservices," in *Proc. IEEE Int. Conf. Services Comput. (SCC)*, Jul. 2018, pp. 253–256.
- [10] D. Escobar, D. Cárdenas, R. Amarillo, E. Castro, K. Garcés, C. Parra, and R. Casallas, "Towards the understanding and evolution of monolithic applications as microservices," in *Proc. 42nd Latin Amer. Comput. Conf. (CLEI)*, Oct. 2016, pp. 1–11.
- [11] R. Chen, S. Li, and Z. Li, "From monolith to microservices: A dataflow-driven approach," in *Proc. 24th Asia-Pacific Softw. Eng. Conf. (APSEC)*, Dec. 2017, pp. 466–475.
- [12] F. H. Vera-Rivera, E. Puerto, H. Astudillo, and C. M. Gaona, "Microservices backlog—A genetic programming technique for identification and evaluation of microservices from user stories," *IEEE Access*, vol. 9, pp. 117178–117203, 2021.
- [13] L. Baresi, M. Garriga, and A. D. Renzis, "Microservices identification through interface analysis," in *Proc. Eur. Conf. Service-Oriented Cloud Comput.*, Jan. 2017, pp. 19–33.
- [14] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction To Natural Language Processing, Computational Linguistics, and Speech Recognition*, 3rd ed., London, U.K.: Pearson, 2019.
- [15] MyBatis. (2020). *MyBatis Jpetstore-6: A Web Application Built on Top of MyBatis 3, Spring 3 and Stripes*. Accessed: Nov. 22, 2020. [Online]. Available: <https://github.com/mybatis/jpetstore-6>
- [16] C. Schröer, F. Kruse, and J. M. Gómez, "A qualitative literature review on microservices identification approaches," in *Proc. 14th Symp. Summer School Service-Oriented Comput. Service-Oriented Comput.* Cham, Switzerland: Springer, Jan. 2020, pp. 151–168.
- [17] S.-P. Ma, Y. Chuang, C.-W. Lan, H.-M. Chen, C.-Y. Huang, and C.-Y. Li, "Scenario-based microservice retrieval using word2 Vec," in *Proc. IEEE 15th Int. Conf. e-Bus. Eng. (ICEBE)*, Xi'an, China, Oct. 2018, pp. 239–244.
- [18] W. Jin, T. Liu, Y. Cai, R. Kazman, R. Mo, and Q. Zheng, "Service candidate identification from monolithic systems based on execution traces," *IEEE Trans. Softw. Eng.*, vol. 47, no. 5, pp. 987–1007, May 2021.
- [19] M. H. Gomes Barbosa and P. H. M. Maia, "Towards identifying microservice candidates from business Rules implemented in stored procedures," in *Proc. IEEE Int. Conf. Softw. Archit. Companion (ICSA-C)*, Salvador, Brazil, Mar. 2020, pp. 41–48.
- [20] S. Li, H. Zhang, Z. Jia, Z. Li, C. Zhang, J. Li, Q. Gao, J. Ge, and Z. Shan, "A dataflow-driven approach to identifying microservices from monolithic applications," *J. Syst. Softw.*, vol. 157, Nov. 2019, Art. no. 110380.
- [21] A. Levcovitz, R. Terra, and M. T. Valente, "Towards a technique for extracting microservices from monolithic enterprise systems," in *Proc. 3rd Workshop Softw. Visualizat., Evol., Maintenance*, Jan. 2016, pp. 1–16.
- [22] M. Gysel, L. Kölbner, W. Giersche, and O. Zimmermann, "Service cutter: A systematic approach to service decomposition," in *Proc. Eur. Conf. Service-Oriented Cloud Comput.*, Jan. 2016, pp. 185–200.
- [23] M. Ahmadvand and A. Ibrahim, "Requirements reconciliation for scalable and secure microservice (De)composition," in *Proc. IEEE 24th Int. Requirements Eng. Conf. Workshops (REW)*, Sep. 2016, pp. 68–73.
- [24] E. Forgey, "Cluster analysis of multivariate data: Efficiency vs. interpretability of classification," *Biometrics*, vol. 21, no. 3, pp. 768–769, 1965.
- [25] L. Kaufman and P. Rousseeuw, "Clustering by means of medoids," in *Statistical Data Analysis Based on the L1-Norm*. Amsterdam, The Netherlands: Elsevier, 1987, pp. 405–416.
- [26] M. Ester, H. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discovery Data Mining*, Jan. 1996, pp. 226–231.
- [27] A. Huang, "Similarity measures for text document clustering," in *Proc. 6th New Zealand Comput. Sci. Res. Student Conf.*, vol. 4, 2008, pp. 9–56.
- [28] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.
- [29] L. Ratnov and D. Roth, "Design challenges and misconceptions in named entity recognition," in *Proc. 13th Conf. Comput. Natural Lang. Learn.*, 2009, pp. 147–155.
- [30] M. Estanol, "Artefact-centric business process models in UML: Specification and reasoning," Ph.D. dissertation, Dept. Service Inf. Syst. Eng., Universitat cnica de Catalunya, Barcelona, Spain, 2016.

- [31] C. Larman, *Applying UML and Patterns: An Introduction To Object-Oriented Analysis and Design and Iterative Development*. Upper Saddle River, NJ, USA: Prentice-Hall, 2004.
- [32] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, Nov. 1987.
- [33] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ, USA: Prentice-Hall, 2003.
- [34] M. Halkidi, Y. Batistakis, and M. Vazirgiannis, "On clustering validation techniques," *J. Intell. Inf. Syst.*, vol. 17, no. 2, pp. 107–1455, 2001.



**IDRIS OUMOUSA** received the bachelor's degree in software engineering from Ibn Zohr University, in 2018, and the master's degree in information systems and intelligent systems from the National Institute of Statistics and Applied Economics (INSEA), Morocco, in 2020. He is currently a Software Engineer with a passion for modernizing IT landscapes, delves into the complexities of microservices architectures at INSEA, as a Ph.D. Researcher. His research interests include microservice architectures, cloud computing, component-based software, artificial intelligence, agile software development, and web development.



**RAJAA SAIDI** received the Ph.D. degree in information systems and software engineering from Mohammed V University in Rabat (UM5), Morocco, and Grenoble Institute of Technology (INPG), France. She is currently a Full Professor of computer science with the National Institute of Statistics and Applied Economics (INSEA), Morocco. She is also a member of the Information Systems, Intelligent Systems and Mathematical Modelling Laboratory (SI2M), INSEA. Her research interests include information systems engineering, business process management, ubiquitous computing, context-aware information systems, service-oriented architectures, and component-based engineering.

...